

Grid Layouts Report

Parker Woolsey

June 6, 2026

Live Site

Here is the URL to my unit-4 grid layouts directory [unit-4 grid layouts directory](#).

Reflections

Layout Tool Decisions

You are building a product page for a client. It needs a hero, a three-column feature section, a full-width testimonial band, and a two-column spec comparison. Walk through which layout tool you would reach for in each section and why. You cannot use the same justification twice.

For the hero section, I would use Flexbox because the content usually needs to be aligned vertically and horizontally in a simple one-dimensional arrangement. A hero often has a heading, paragraph, button, and maybe an image, so Flexbox would make it easy to control spacing and alignment.

For the three-column feature section, I would use CSS Grid with `repeat(3, 1fr)` or a responsive auto-fit pattern. Grid is a good choice here because the section is organized as equal columns, and the layout is more about the structure of the page than the size of the individual content pieces.

For the full-width testimonial band, I would use normal block layout with a constrained inner container. The band itself should stretch across the full viewport, but the testimonial text should stay readable inside a max-width container.

For the two-column spec comparison, I would use CSS Grid because the content needs a clear side-by-side comparison. Grid makes it easier to line up related information across columns and keep the comparison visually balanced.

Responsive Areas

Your client comes back and says they want the editorial sidebar to move below the article on tablet but above it on mobile. What changes, and what stays the same? What does that tell you about how grid-template-areas distributes responsibility in a responsive layout?

If the sidebar needs to move below the article on tablet but above it on mobile, I would change the `grid-template-areas` at different breakpoints. The HTML structure could stay the same, and the class names could stay the same. The grid container would still control the placement of the article and sidebar.

For example, mobile could use:



```
1 grid-template-areas:  
2 "sidebar"  
3 "main";
```

Tablet could use:



```
1 grid-template-areas:  
2 "main"  
3 "sidebar";
```

Desktop could use:



```
1 grid-template-areas:  
2 "main sidebar";
```

This shows that `grid-template-areas` separates the document structure from the visual layout. The HTML keeps the content meaningful and organized, while CSS takes responsibility for where each area appears at different screen sizes.

Mentoring a Junior Developer

A junior developer on your team always reaches for `auto-fill` instead of `auto-fit` because it is simpler to remember one keyword. What advice would you give them?

I would tell the junior developer that auto-fill and auto-fit are similar, but they are not interchangeable. auto-fill keeps creating empty tracks even when there is not enough content to fill them. auto-fit collapses those empty tracks so the existing items can stretch and use the available space.

My advice would be to start with auto-fit for most responsive card grids because it usually creates the behavior people expect: cards expand to fill the row naturally. Use auto-fill when you actually want the grid to preserve empty column space, such as when maintaining a strict grid structure matters more than stretching the items.